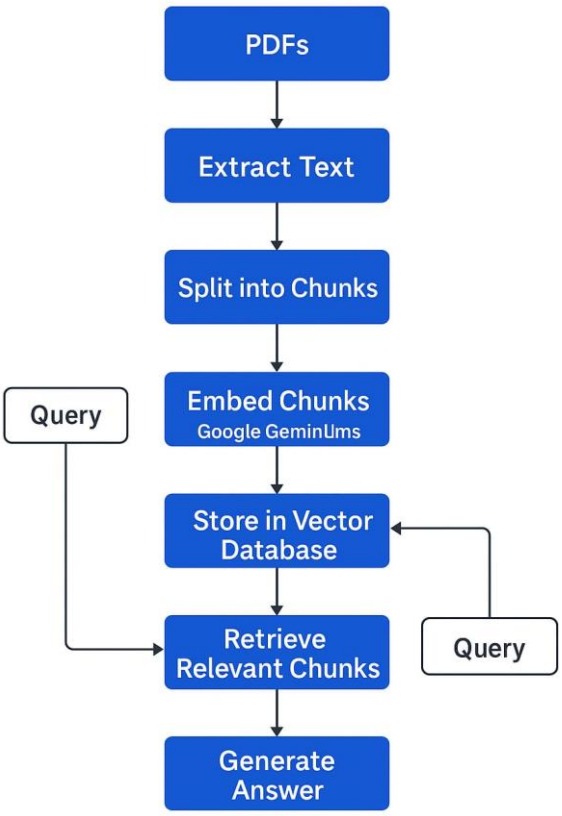
📚Medico Doc Q&A System-Documentation

Overview:
This project is a **Retrieval-Augmented Generation (RAG)** system designed to answer natural language questions from a set of **medical PDFs**. It uses:

- **LangChain framework** to build the RAG pipeline
- **Google Gemini Pro (via LangChain)** for the language model
- **ChromaDB** for vector storage
- **Gradio** for user interaction through a web UI



Retrievial-Augmented RAG

Component Breakdown
1. Document Loader

- **Module**: fitz (PyMuPDF)
- **Function**: Reads PDFs from ./data directory and extracts text page by page.

2. Text Splitter

- **Tool**: RecursiveCharacterTextSplitter from LangChain
- **Purpose**: Splits long texts into smaller overlapping chunks to preserve context for retrieval.

3. Embeddings

- **Tool**: GoogleGenerativeAIEmbeddings
- **Model**: models/embedding-001
- **Purpose**: Converts text chunks into dense vectors suitable for semantic similarity search.

4. Vector Store

- **Tool**: Chroma (LangChain Community)
- **Usage**: Stores the embedding vectors and supports fast retrieval via similarity search.

5. Retriever

- **Function**: Wraps Chroma to retrieve top k=4 most relevant chunks for a given question.

6. LLM Integration

- **Tool**: ChatGoogleGenerativeAI from LangChain
- **Model**: gemini-2.5-pro-preview-03-25
- **Function**: Takes query + retrieved context and generates a concise answer.

7. QA Chain

- **Tool**: RetrievalQA.from_chain_type
- **Integration**: Connects retriever and LLM into a single callable interface.

8. Frontend (UI)

- **Tool**: Gradio
- **Interface**: User types a query and gets an answer + source references.